National University of Computer and Emerging Sciences

Personal Finance Tracker and Budget Management System

by

(24L-0620 Talha Ahmad)
(24L-0630 Muhammad Ibrahim)
(24L-0631 Muhammad Aariz)

Instructor	Ahmed Jaan
Lab TA	Shahzeb Mubashar
Semester	Spring 2026

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

1. Project Overview

2. Database Schema Design

3. Table Descriptions

4. Relationships and Constraints

5. Sample Queries

6. Schema Diagrams

7. Conclusion

1. Project Overview

The Personal Finance Tracker and Budget Management System is a comprehensive database solution designed to help users manage their personal finances effectively. The system enables users to track income and expenses, manage multiple financial accounts, set category-wise budgets, and generate analytical reports for better financial decision-making.

1.1 Objectives

The primary objective is to design and implement a secure and efficient database system that supports complete personal finance management. Key goals include maintaining data integrity, ensuring user privacy, providing flexible transaction categorization, and enabling comprehensive financial reporting.

1.2 Key Features

Feature	Description
User Management	Secure authentication and isolated user data
Multi-Account Support	Bank accounts, cash, digital wallets, credit cards
Transaction Tracking	Comprehensive income and expense recording
Budget Planning	Category-wise budget limits with alerts
Recurring Transactions	Support for salary, rent, subscriptions
Financial Reports	Monthly summaries and spending analysis

2. Database Schema Design

The database schema consists of five interconnected tables designed to maintain data integrity while providing flexibility for personal finance management. The design follows normalization principles to minimize redundancy and ensure efficient data storage and retrieval.

2.1 Schema Overview

Table Name	Purpose	Key Features
Users	User accounts	Authentication, profile data
Categories	Transaction types	Income/Expense classification
Accounts	Financial accounts	Multiple account types, balances
Transactions	Financial records	Income, expenses, transfers
Budgets	Spending limits	Category-wise monthly budgets

2.2 Design Principles

Data Integrity: All tables use auto-incrementing primary keys and foreign key constraints to maintain referential integrity. Check constraints ensure data validity (e.g., positive amounts).

User Isolation: Each user's data is completely isolated through foreign key relationships, ensuring privacy and security.

Flexibility: The schema supports multiple account types, custom categories, and both one-time and recurring transactions.

Normalization: Data is organized to minimize redundancy while maintaining query efficiency.

3. Table Descriptions

3.1 Users Table

The Users table stores account information and authentication details for each user. It serves as the central table with which all other tables establish foreign key relationships.

Field	Type	Description
user_id	INT (PK)	Unique identifier, auto-increment
username	VARCHAR(50)	Login username, unique
email	VARCHAR(100)	Email address, unique
password_hash	VARCHAR(255)	Hashed password for security
full_name	VARCHAR(100)	User's full name
created_at	TIMESTAMP	Account creation date
last_login	TIMESTAMP	Last login timestamp

3.2 Categories Table

The Categories table organizes transactions into user-defined categories (e.g., Groceries, Salary, Entertainment). Each category is classified as either Income or Expense.

Field	Type	Description
category_id	INT (PK)	Unique identifier, auto-increment
user_id	INT (FK)	References Users table
category_name	VARCHAR(50)	Name of category
category_type	ENUM	'Income' or 'Expense'
description	VARCHAR(255)	Optional category description

3.3 Accounts Table

The Accounts table manages multiple financial accounts for each user, including bank accounts, cash, digital wallets, and credit cards. It tracks current balances for each account.

Field	Type	Description
account_id	INT (PK)	Unique identifier, auto-increment
user_id	INT (FK)	References Users table
account_name	VARCHAR(100)	Name of account
account_type	ENUM	Cash, Bank, Digital Wallet, etc.
current_balance	DECIMAL(15,2)	Current account balance
created_at	TIMESTAMP	Account creation date
is_active	BOOLEAN	Whether account is active

3.4 Transactions Table

The Transactions table is the core of the system, recording all financial activities including income, expenses, and transfers. It supports both one-time and recurring transactions.

Field	Type	Description
transaction_id	INT (PK)	Unique identifier
user_id	INT (FK)	References Users
account_id	INT (FK)	References Accounts
category_id	INT (FK)	References Categories
transaction_type	ENUM	Income, Expense, Transfer
amount	DECIMAL(15,2)	Transaction amount (positive)
transaction_date	DATE	Date of transaction
description	VARCHAR(255)	Optional notes
payment_method	VARCHAR(50)	Cash, Card, Transfer, etc.
is_recurring	BOOLEAN	If transaction repeats
recurring_frequency	ENUM	Daily, Weekly, Monthly, Yearly

3.5 Budgets Table

The Budgets table allows users to set spending limits for different expense categories on a monthly basis. It includes alert thresholds to notify users when approaching their limits.

Field	Type	Description
budget_id	INT (PK)	Unique identifier
user_id	INT (FK)	References Users
category_id	INT (FK)	References Categories
budget_amount	DECIMAL(15,2)	Budget limit amount
budget_month	DATE	Month for budget
alert_threshold	DECIMAL(5,2)	Alert percentage (0-100)
created_at	TIMESTAMP	Record creation date

4. Relationships and Constraints

4.1 Entity Relationships

The database schema implements a well-defined set of relationships between tables to maintain data integrity and support complex queries. All relationships follow the one-to-many (1:N) pattern, with the Users table serving as the parent entity for most tables.

Parent Table	Child Table	Relationship	Description
Users	Categories	1:N	Each user can have multiple categories
Users	Accounts	1:N	Each user can have multiple accounts
Users	Transactions	1:N	Each user can make multiple transactions
Users	Budgets	1:N	Each user can set multiple budgets
Categories	Transactions	1:N	Each category can have multiple transactions
Accounts	Transactions	1:N	Each account can have multiple transactions
Categories	Budgets	1:N	Each category can have multiple budgets

4.2 Data Integrity Constraints

The schema implements multiple types of constraints to ensure data quality and consistency:

Constraint Type	Implementation	Purpose
Primary Keys	Auto-incrementing INT	Unique identification of records
Foreign Keys	ON DELETE CASCADE/RESTRICT	Maintain referential integrity
Unique Constraints	Username, Email, etc.	Prevent duplicate values
Check Constraints	Amount > 0, Threshold 0-100	Validate data ranges
NOT NULL	Required fields	Ensure mandatory data exists
ENUM Types	Predefined values	Restrict to valid options

4.3 Cascade Operations

ON DELETE CASCADE: Applied to user-owned data (Categories, Accounts, Transactions, Budgets). When a user account is deleted, all associated records are automatically removed to maintain data consistency.

ON DELETE RESTRICT: Applied to referenced data (Accounts and Categories in Transactions). Prevents deletion of accounts or categories that have associated transactions, protecting historical data integrity.

5. Sample Queries

The following queries demonstrate common operations supported by the database schema. These use basic SQL features including JOINs and subqueries.

5.1 View User Transactions with Details

This query retrieves all transactions for a specific user with complete category and account information:

```
SELECT t.transaction_id, t.amount, t.transaction_date, c.category_name,
a.account_name, t.description FROM Transactions t JOIN Categories c ON
t.category_id = c.category_id JOIN Accounts a ON t.account_id = a.account_id
WHERE t.user_id = 1 ORDER BY t.transaction_date DESC;
```

5.2 Calculate Monthly Income and Expenses

This query calculates total income and expenses for a specific month:

```
SELECT SUM(CASE WHEN transaction_type = 'Income' THEN amount ELSE 0 END) as
total_income, SUM(CASE WHEN transaction_type = 'Expense' THEN amount ELSE 0 END)
as total_expenses FROM Transactions WHERE user_id = 1 AND transaction_date >=
'2026-02-01' AND transaction_date < '2026-03-01';
```

5.3 Check Budget vs Actual Spending

This query compares budgeted amounts with actual spending for each category:

```
SELECT c.category_name, b.budget_amount, COALESCE(SUM(t.amount), 0) as
total_spent, b.budget_amount - COALESCE(SUM(t.amount), 0) as remaining FROM
Budgets b JOIN Categories c ON b.category_id = c.category_id LEFT JOIN
Transactions t ON b.category_id = t.category_id AND t.transaction_type =
'Expense' WHERE b.user_id = 1 AND b.budget_month = '2026-02-01' GROUP BY
c.category_name, b.budget_amount;
```

5.4 Find Top Spending Categories

This query identifies the categories with the highest spending using a subquery:

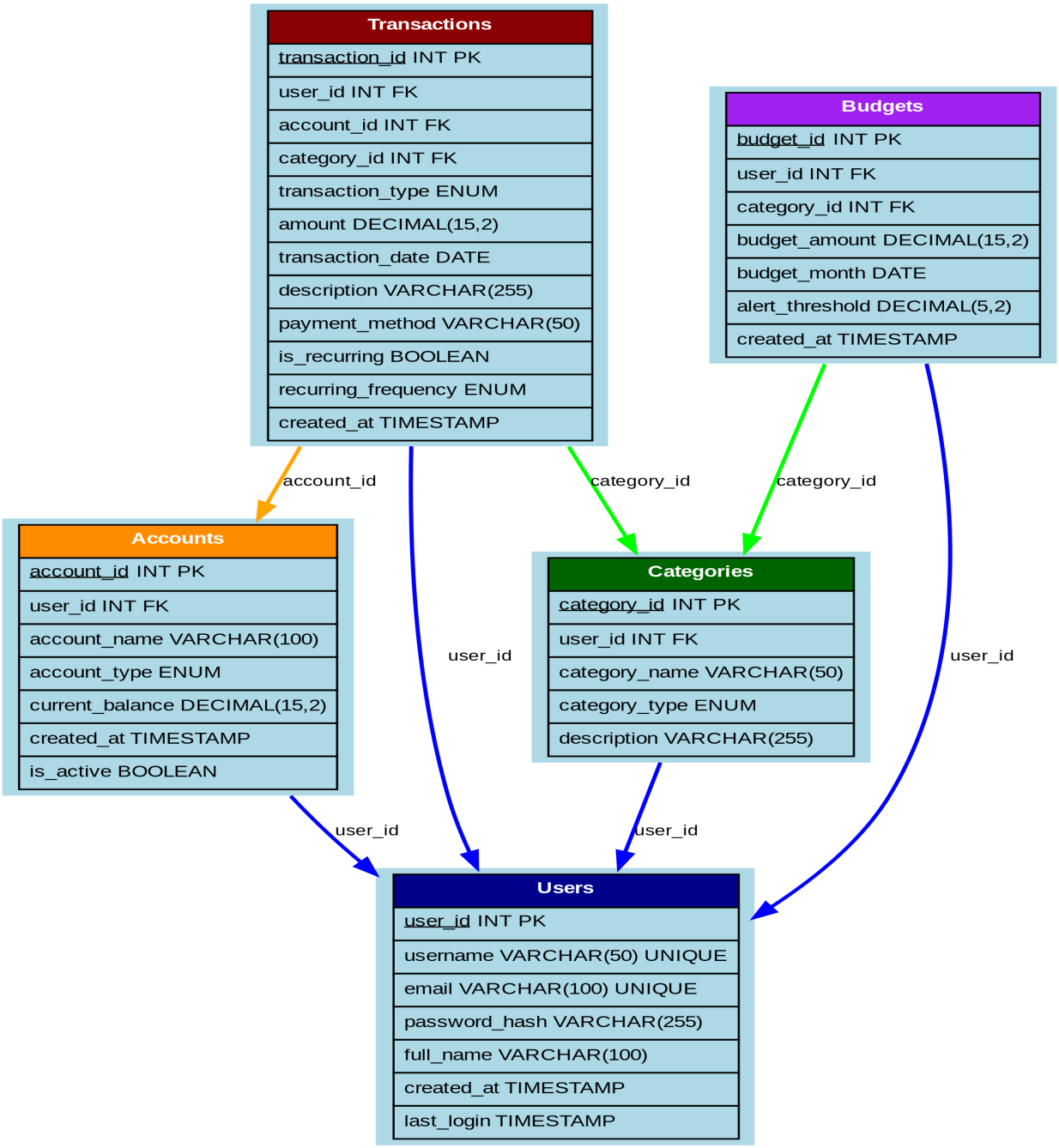
```
SELECT c.category_name, SUM(t.amount) as total_spent FROM Transactions t JOIN
Categories c ON t.category_id = c.category_id WHERE t.user_id = 1 AND
t.transaction_type = 'Expense' AND t.transaction_date >= (SELECT
DATE_SUB(CURDATE(), INTERVAL 1 MONTH)) GROUP BY c.category_name ORDER BY
total_spent DESC LIMIT 5;
```

6. Schema Diagrams

The following diagrams provide visual representations of the database schema, showing tables, relationships, and key constraints.

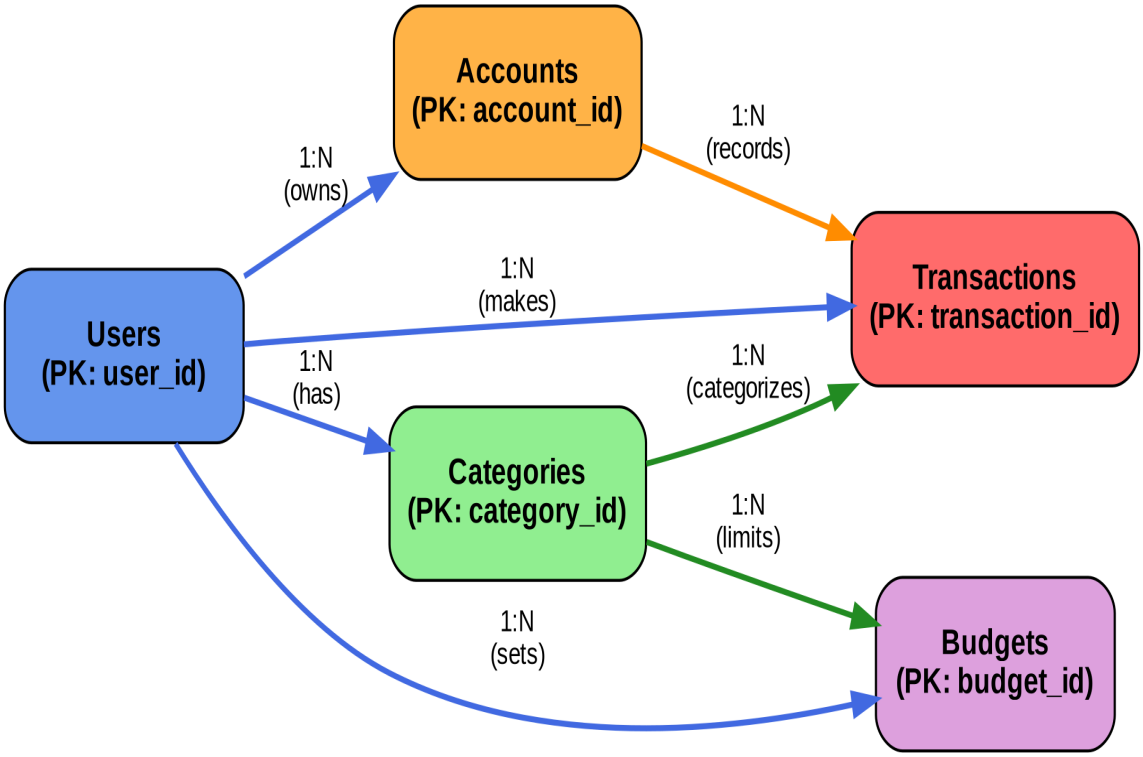
6.1 Detailed ER Diagram

This diagram shows all tables with complete column details, data types, and foreign key relationships:



6.2 Simplified Relationship Diagram

This diagram focuses on table relationships and cardinality, showing how entities are connected:



7. Conclusion

7.1 Summary

The Personal Finance Tracker database schema provides a solid foundation for comprehensive personal finance management. The design successfully balances simplicity with functionality, using only basic database concepts while supporting all required features including user authentication, multi-account management, transaction tracking, budget planning, and financial reporting.

7.2 Key Achievements

Data Integrity: The schema employs comprehensive constraints including primary keys, foreign keys, unique constraints, and check constraints to ensure data quality.

Scalability: The design supports multiple users with complete data isolation, allowing the system to grow without architectural changes.

Flexibility: Users can define custom categories, manage multiple account types, and track both recurring and one-time transactions.

Simplicity: The schema uses only fundamental database concepts (tables, joins, subqueries) making it maintainable and easy to understand.

7.3 Future Enhancements

Potential additions for subsequent deliverables include savings goals tracking, investment portfolio management, shared expense splitting, bill payment reminders, and advanced analytics with data visualization. The current schema design provides a solid foundation for these extensions.

This deliverable demonstrates a well-structured database schema that meets all project objectives while maintaining simplicity and clarity. The design is ready for implementation and testing in the next phase of the project.